

操作系统原理

PRINCIPLES OF
OPERATING SYSTEM

第7讲

存储模型1

存储管理

- ◎ 基本概念
- ◎ 物理内存管理
- ◎ 伙伴系统
- ◎ 基本内存管理方案
- ◎ 交换技术 (Swapping)

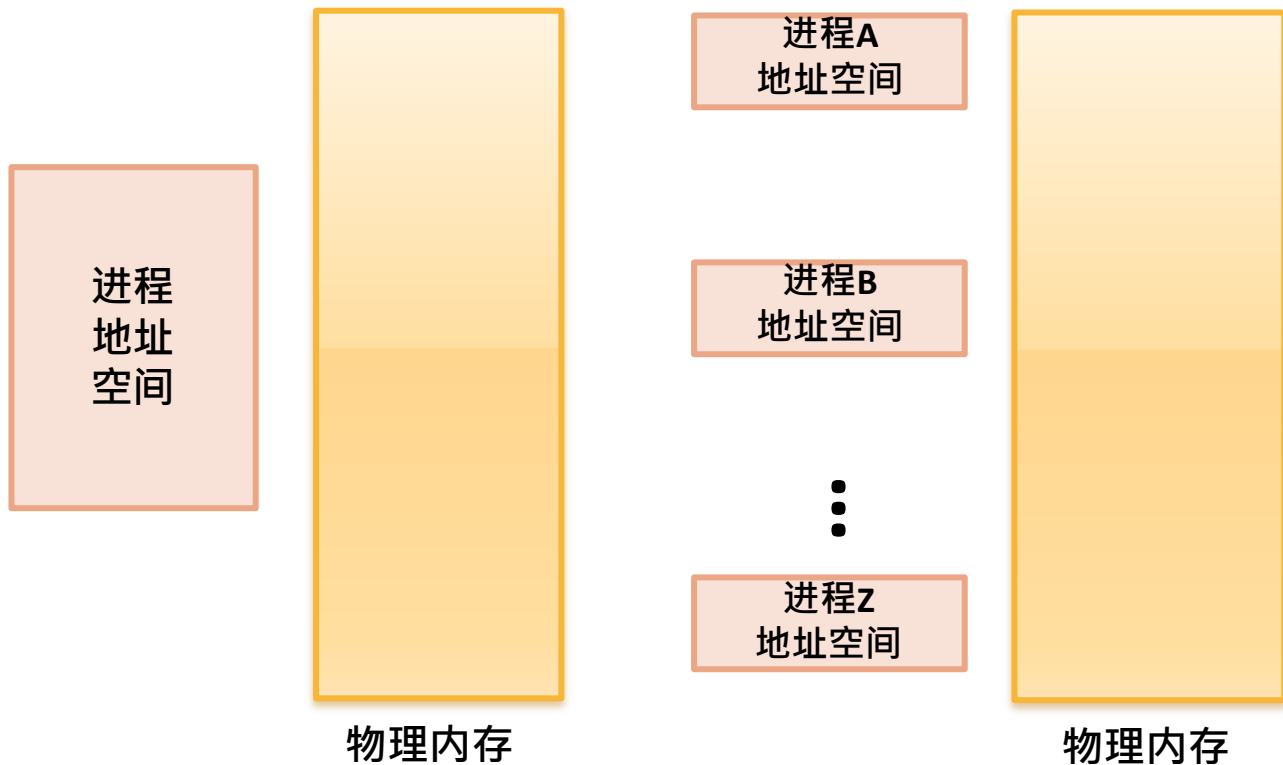
地址转换、地址映射、地址翻译

基本概念：地址重定位 RELOCATION

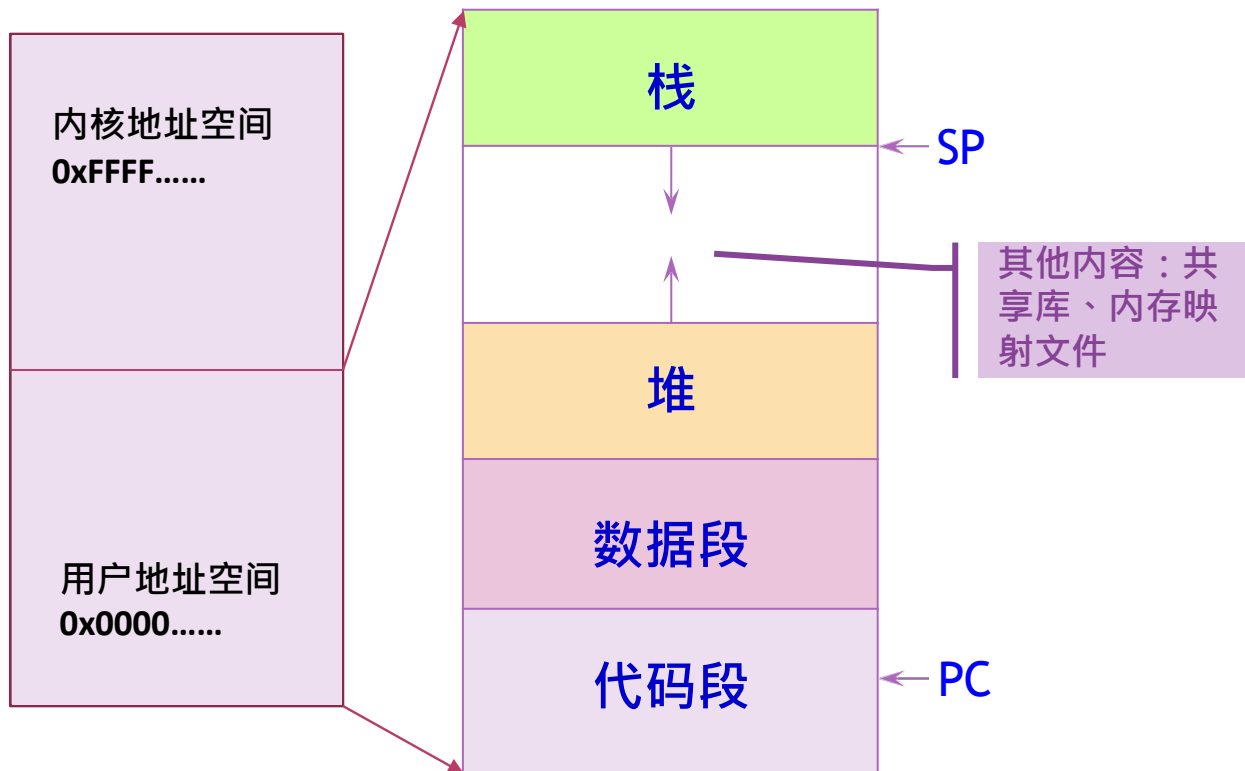
.....已经了解的

- 程序装载到内存才可以运行
通常，程序以可执行文件格式保存在磁盘上
- 多道程序设计模型
允许多个程序同时进入内存
- 每个进程有自己的地址空间
一个进程执行时不能访问另一个进程的地址空间
进程不能执行不适合的操作

要解决的问题



复习：进程地址空间



讨论

- 进程中的地址不是最终的物理地址
- 在进程运行前无法计算出物理地址
因为：不能确定进程被加载到内存什么地方

→→ 需要 地址重定位 的支持

地址转换、地址变换、地址翻译、地址映射
Translation、Mapping

地址重定位

- 逻辑地址（相对地址，虚拟地址）

用户程序经过编译、汇编后形成目标代码，目标代码通常采用相对地址的形式，其首地址为0，其余地址都相对于首地址而编址

不能用逻辑地址在内存中读取信息

- 物理地址（绝对地址，实地址）

内存中存储单元的地址 可直接寻址

为了保证CPU执行指令时可正确访问内存单元，需
要将用户程序中的逻辑地址转换为运行时可由机器
直接寻址的物理地址，这一过程称为地址重定位

静态重定位与动态重定位

静态重定位：

当用户程序加载到内存时，一次性实现逻辑地址到物理地址的转换

 一般可以由软件完成

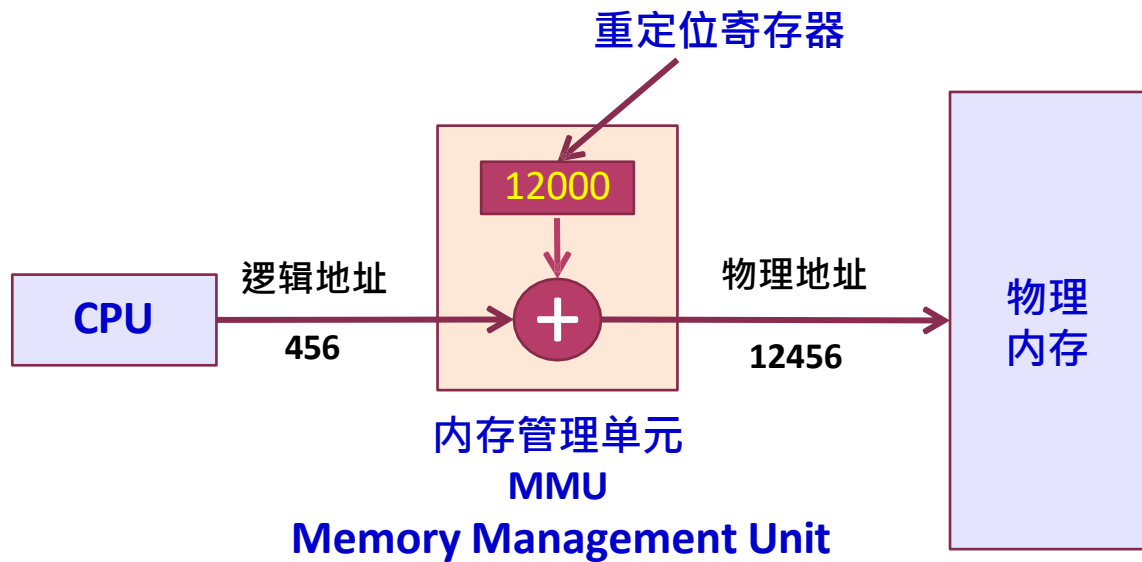
动态重定位：

在进程执行过程中进行地址变换

→ → 即逐条指令执行时完成地址转换

 需要硬件部件支持

动态重定位实现



位图法、空闲区表、空闲区链表

物理内存管理

空闲内存管理

● 数据结构

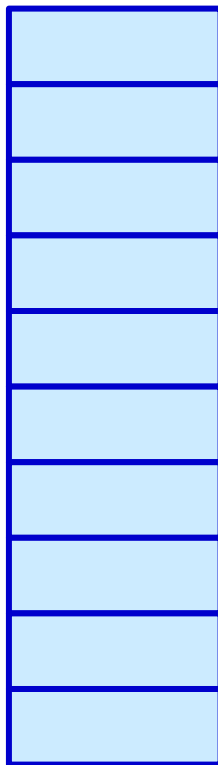
➤ 位图

- 每个分配单元对应于位图中的一位，0表示空闲，1表示占用（或者相反）

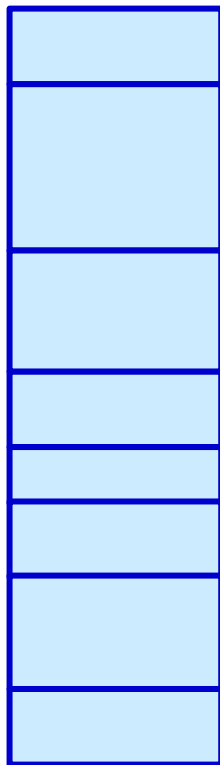
➤ 空闲区表、已分配区表

- 表中每一项记录了空闲区（或已分配区）的起始地址、长度、标志

➤ 空闲块链表



等长划分



不等长划分

内存分配算法

- 首次适配 **first fit**

在空闲区表中找到第一个满足进程要求的空闲区

- 下次适配 **next fit**

从上次找到的空闲区处接着查找

- 最佳适配 **best fit**

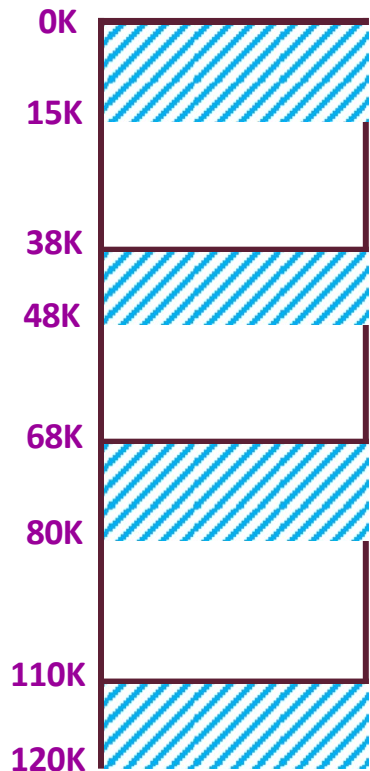
查找整个空闲区表，找到能够满足进程要求的最小空闲区

- 最差适配 **worst fit**

总是分配满足进程要求的最大空闲区

将该空闲区分为两部分，一部分供进程使用，另一部分形成新的空闲区

示例



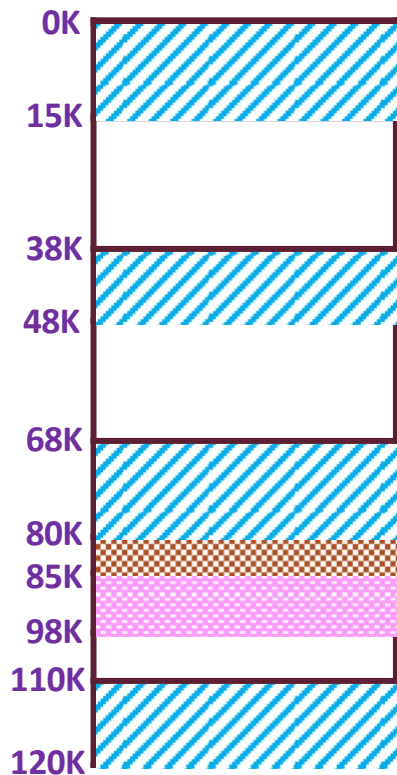
空闲区表

始址	长度	标志
15K	23K	未分配
48K	20K	未分配
80K	30K	未分配
		空项
		空项

已分配区表

始址	长度	标志
0K	15K	P ₁
38K	10K	P ₂
68K	12K	P ₃
110K	10K	P ₄
		空项
		空项

示例



空闲区表

始址	长度	标志
15K	23K	未分配
48K	20K	未分配
98K	12K	未分配
		空项
		空项

已分配区表

始址	长度	标志
0K	15K	P ₁
38K	10K	P ₂
68K	12K	P ₃
110K	10K	P ₄
80K	5K	P ₅
85K	13K	P ₆

回收问题

- 内存回收算法

当某一块归还后，前后空闲空间合并，修改内存空闲区表

四种情况

上相邻、下相邻、上下都相邻、上下都不相邻

伙伴系统 BUDDY SYSTEM

伙伴系统

Linux低层内存管理采用

一种特殊的“
分离适配”
算法

- 一种经典的内存分配方案
- 主要思想：将内存按2的幂进行划分，组成若干空闲块链表；查找该链表找到能满足进程需求的最佳匹配块
- 算法：
 - 首先将整个可用空间看作一块： 2^U
 - 假设进程申请的空间大小为 s ，如果满足 $2^{U-1} < s \leq 2^U$ ，则分配整个块
 - 否则，将块划分为两个大小相等的伙伴，大小为 2^{U-1}
 - 一直划分下去直到产生**大于或等于 s 的最小块**

伙伴系统的例子



单一连续区、固定分区、可变分区

基本内存管理方案1

整个进程进入内存中一片
连续区域

基本内存管理方案



单一连续区

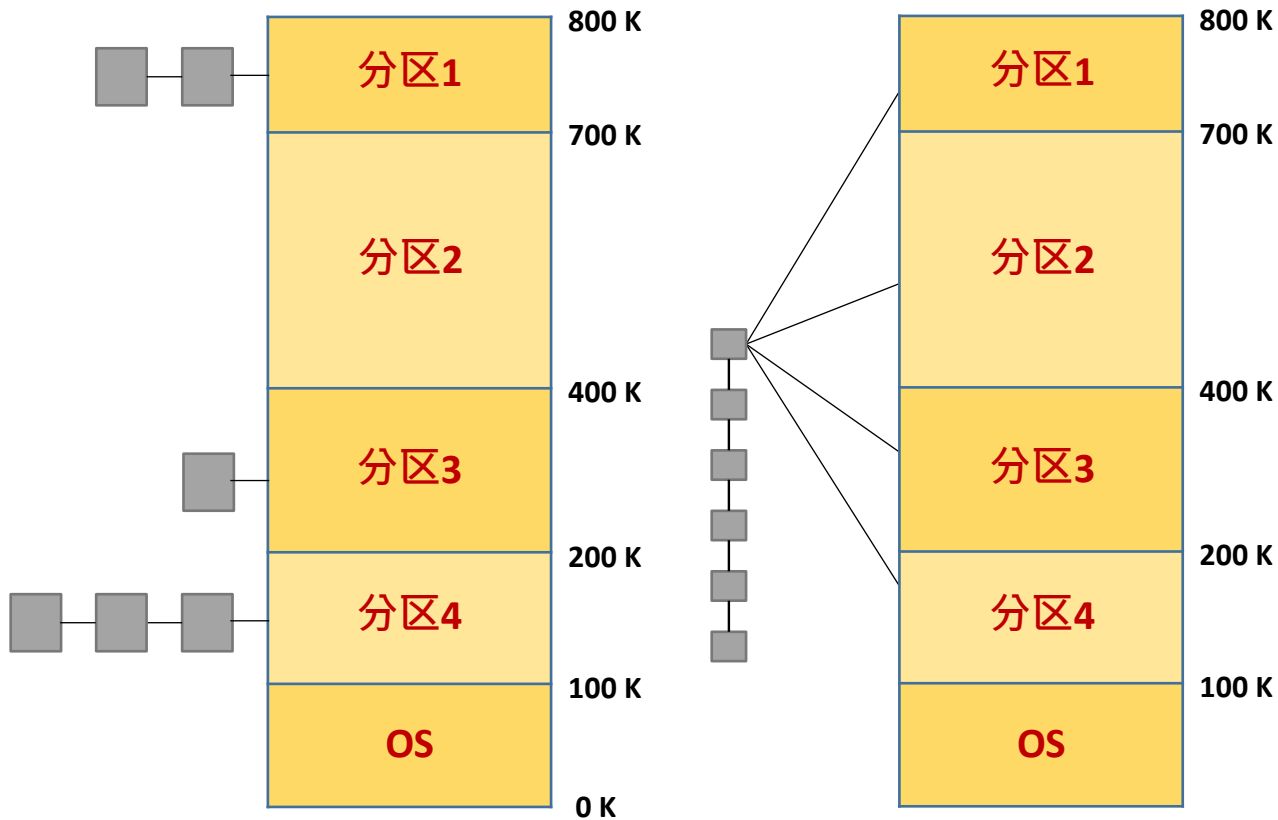
特点：一段时间内只有一个进程在内存
简单，内存利用率低



固定分区

- 把内存空间分割成若干区域，称为分区
- 每个分区的大小可以相同也可以不同
- 分区大小固定不变
- 每个分区装一个且只能装一个进程

固定分区示例



可变分区



- 根据进程的需要，把内存空闲空间分割出一个分区，分配给该进程
- 剩余部分成为新的空闲区



碎片问题解决

- ◎ 碎片 → 很小的、不易利用的空闲区
→ 导致内存利用率下降
- ◎ 解决方案 → 紧缩技术 (memory compaction)
在内存移动程序，将所有小的空闲区合并为较大的空闲区
又称：压缩技术，紧致技术，搬家技术
- ◎ 紧缩时要考虑的问题
系统开销？ 移动时机？

页式、段式、段页式

基本内存管理方案2

一个进程进入内存中若干片
不连续的区域

页式存储管理方案

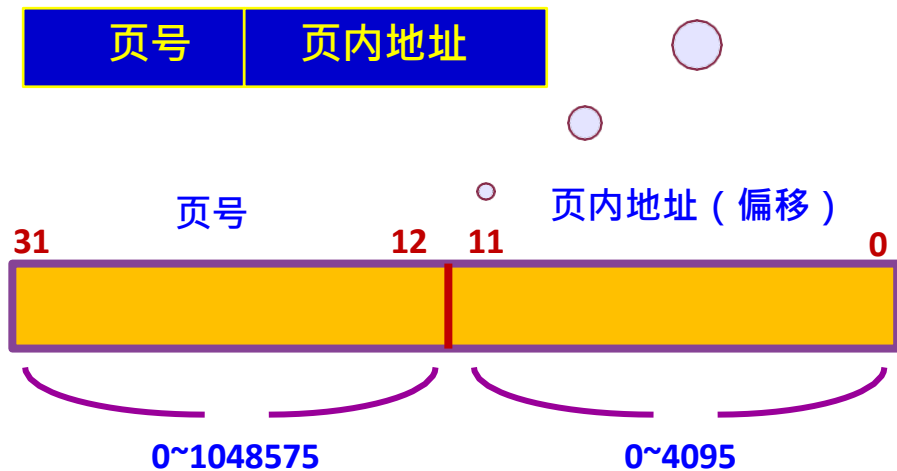
◎ 设计思想

- 用户进程地址空间被划分为大小相等的部分，称为页 (page) 或页面，从0开始编号
- 内存空间按同样大小划分为大小相等的区域，称为页框 (page frame)，从0开始编号；也称为物理页面，页帧，内存块
- 内存分配 (规则)
以页为单位进行分配，并按进程需要的页数来分配；逻辑上相邻的页，物理上不一定相邻
- 典型页面尺寸：**4K 或 4M**

页式存储管理方案

划分是由系统自动完成的，对用户是透明的

逻辑地址

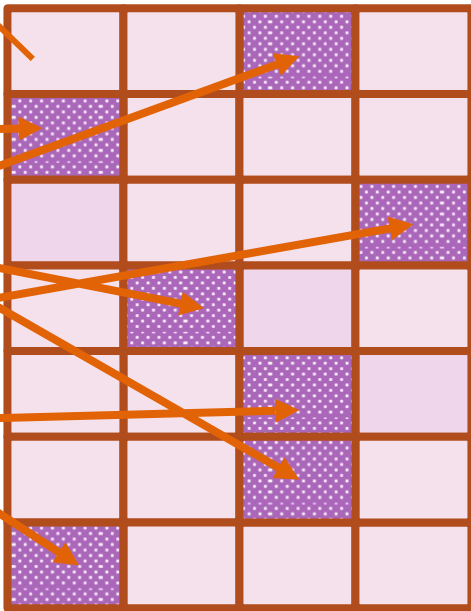


内存分配

页框
(页帧、物理页面、
内存块)

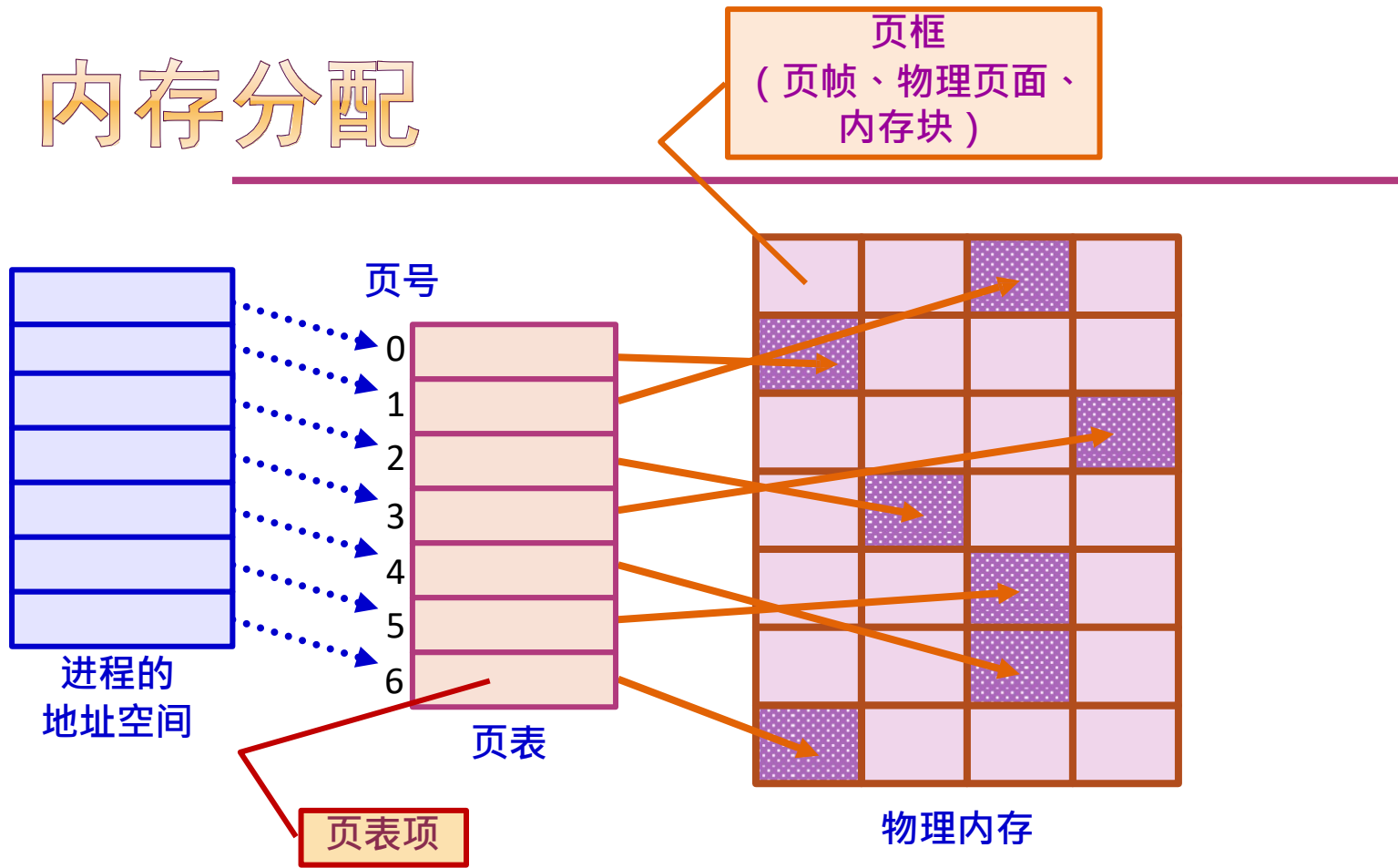


进程的
地址空间



物理内存

内存分配



相关数据结构及地址转换

◎ 页表

- **页表项**：记录了逻辑页号与页框号的对应关系
- 每个进程一个页表，存放在内存
- 页表起始地址保存在何处？

内碎片

◎ 空闲内存管理

◎ 地址转换（硬件支持）

CPU取到逻辑地址，自动划分为页号和页内地址；
用页号查页表，得到页框号，再与页内偏移拼接成为物理地址

页号

页内地址

段式存储管理方案

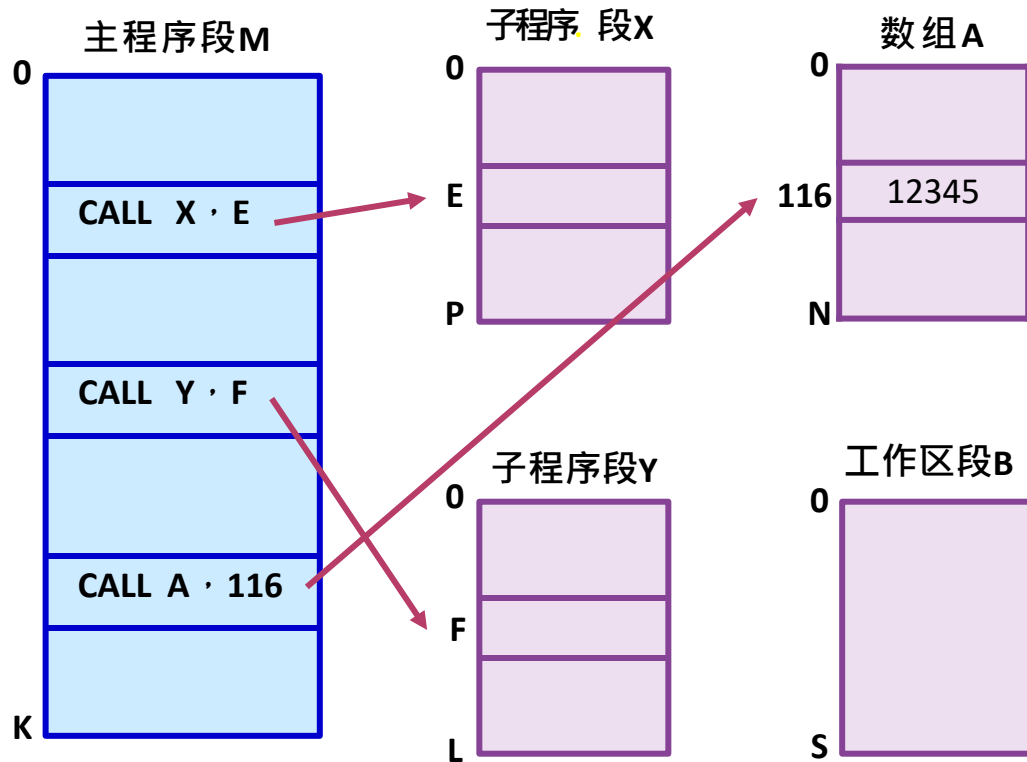
◎ 设计思想

- 用户进程地址空间：按程序自身的逻辑关系划分为若干个程序段，每个程序段都有一个段名
- 内存空间被动态划分为若干长度不相同的区域，称为物理段，每个物理段由起始地址和长度确定
- 内存分配（规则）：以段为单位进行分配，每段在内存中占据连续空间，但各段之间可以不相邻

◎ 逻辑地址

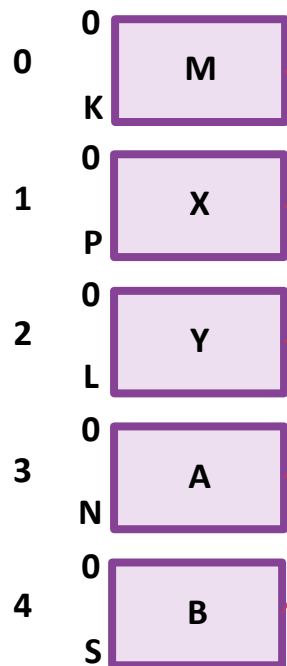
段号	段内地址
----	------

示意图(1/2)



示意图(2/2)

逻辑段号



进程的地址空间

长度	段地址
0 K	3200
1 P	1500
2 L	6000
3 N	8000
4 S	5000

段表

1000

1500

3200

5000

6000

8000

操作系统

P

K

S

L

N

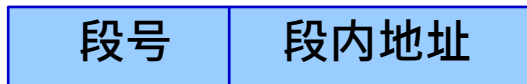
物理内存

相关数据结构及地址转换

- 段表

- 每项记录了段号、段首地址和段长度之间的关系
- 每个进程一个段表，存放在内存
- 段表起始地址保存在何处？

- 物理内存管理



- 地址转换（硬件）

CPU取到逻辑地址，用段号查段表，得到该段在内存的起始地址，与段内偏移地址计算出物理地址

段页式存储管理方案

- 产生背景

综合页式、段式方案的优点，克服二者的缺点

- 设计思想

用户进程划分：

先按段划分，每一段再按页面划分

逻辑地址：

段号	段内地址	
	页号	页内地址

内存划分：同页式存储管理方案

内存分配：以页为单位进行分配

段页式存储管理

◎ 数据结构及有关操作

- 段表：记录了每一段的页表始址和页表长度
- 页表：记录了逻辑页号与页框号的对应关系
每一段有一张页表，一个进程有多个页表
- 空闲区管理：同页式管理
- 内存分配、回收：同页式管理

◎ 地址转换

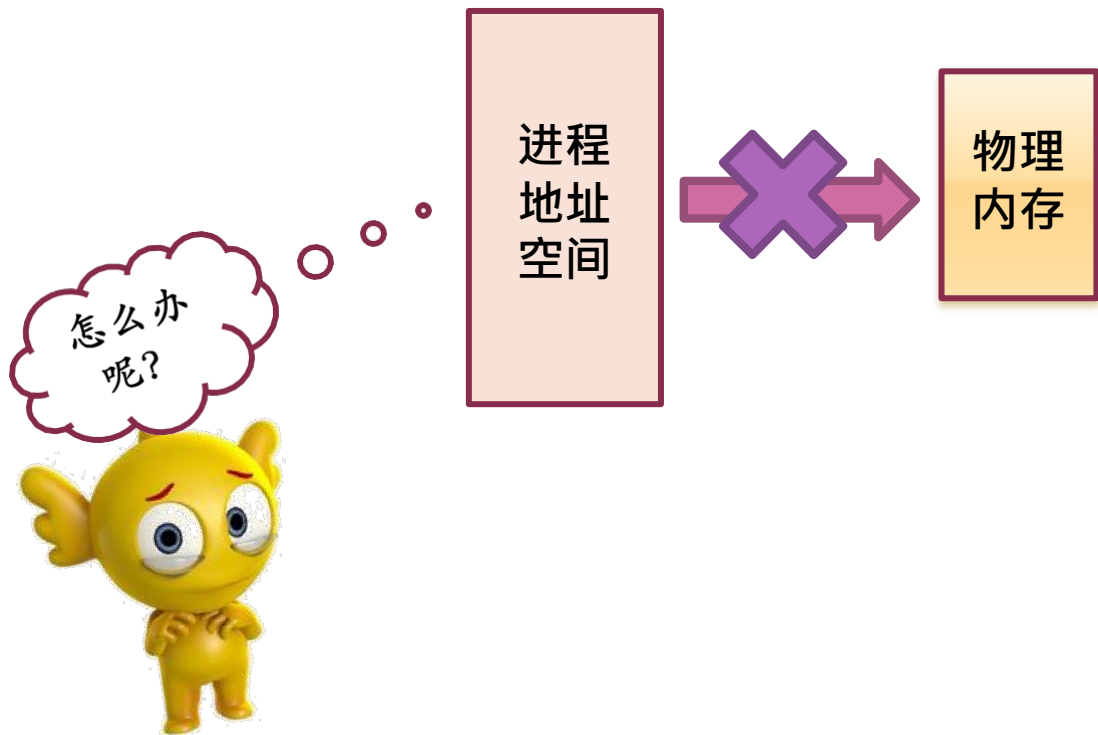
段号	段内地址	
	页号	

基本内存管理方案小结

单一连续区	每次只运行一个用户程序，用户程序独占内存，它总是被加载到同一个内存地址上
固定分区	把可分配的内存空间分割成若干个连续区域，每一区域称为分区。每个分区的大小可以相同也可以不同，分区大小固定不变，每个分区装一个且只能装一个进程
可变分区	根据进程的需求，把可分配的内存空间分割出一个分区，分配给该进程
页式	把用户程序地址空间划分成大小相等的部分，称为页。内存空间按页的大小划分为大小相等的区域，称为内存块（物理页面，页框，页帧）。以页为单位进行分配，逻辑上相邻的页，物理上不一定相邻
段式	用户程序地址空间按进程自身的逻辑关系划分为若干段，内存空间被动态的划分为若干个长度不相同的区域（可变分区）。以段为单位分配内存，每一段在内存中占据连续空间，各段之间可以不连续存放
段页式	用户程序地址空间：段式；内存空间：页式；分配单位：页

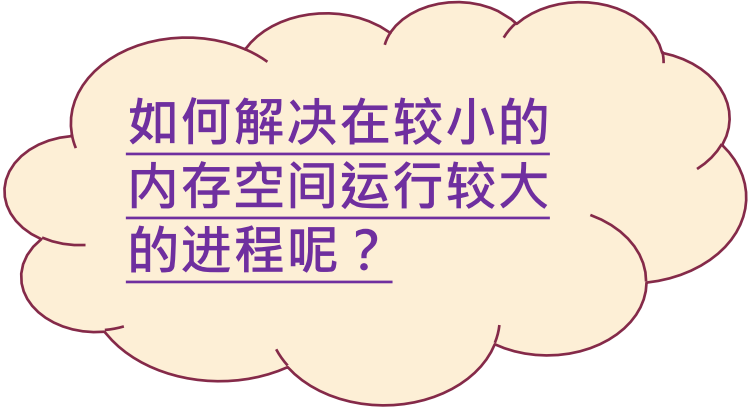
交换技术 *SWAPPING*

内存不足时如何管理？



内存“扩充”技术

- ◎ 内存紧缩技术（例如：可变分区）
- ◎ 覆盖技术 **overlaying**
- ◎ 交换技术 **swapping**
- ◎ 虚拟存储技术 **virtual memory**



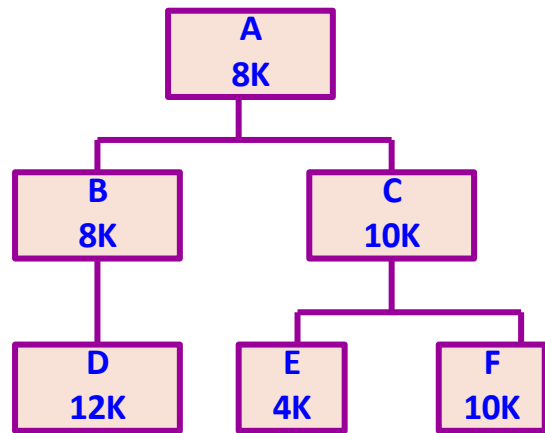
如何解决在较小的
内存空间运行较大
的进程呢？

覆盖技术(OVERLAYING)

- 解决的问题 → 程序大小超过物理内存总和
- 程序执行过程中，程序的不同部分在内存中相互替代
 - 按照其自身的逻辑结构，将那些不会同时执行的程序段共享同一块内存区域
 - 要求程序各模块之间有明确的调用结构
- 程序员声明覆盖结构，操作系统完成自动覆盖

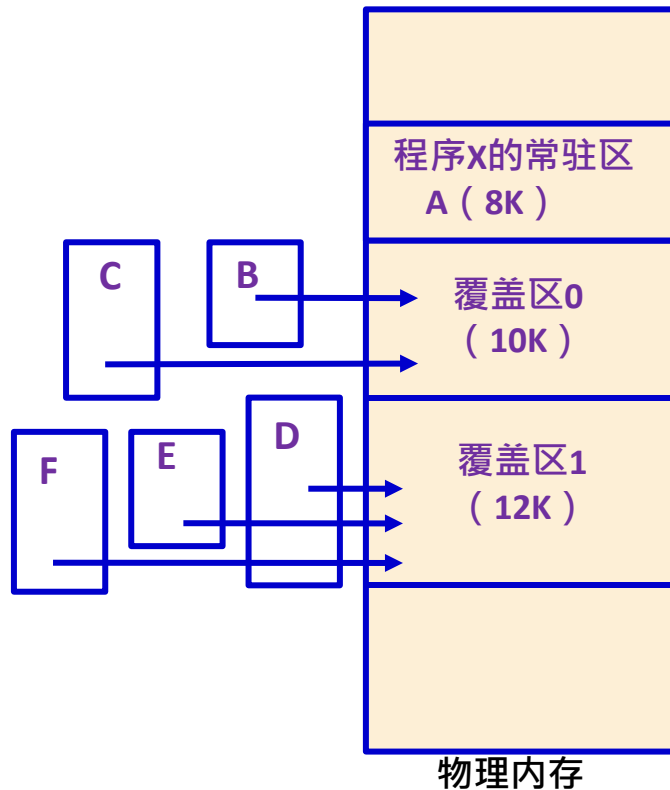
主要用于早期的操作系统

覆盖技术示例



程序X的调用结构

缺点：对用户
不透明，增加
了用户负担



交换技术(SWAPPING)

● 设计思想

内存空间紧张时，系统将内存中某些进程暂时移到外存，把外存中某些进程换进内存，占据前者所占用的区域（进程在内存与磁盘之间的动态调度）

● 讨论：实现时遇到的问题

- ✓ 进程的哪些内容要交换到磁盘？会遇到什么困难？
- ✓ 在磁盘的什么位置保存被换出的进程？
- ✓ 交换时机？
- ✓ 如何选择被换出的进程？
- ✓ 如何处理进程空间增长？

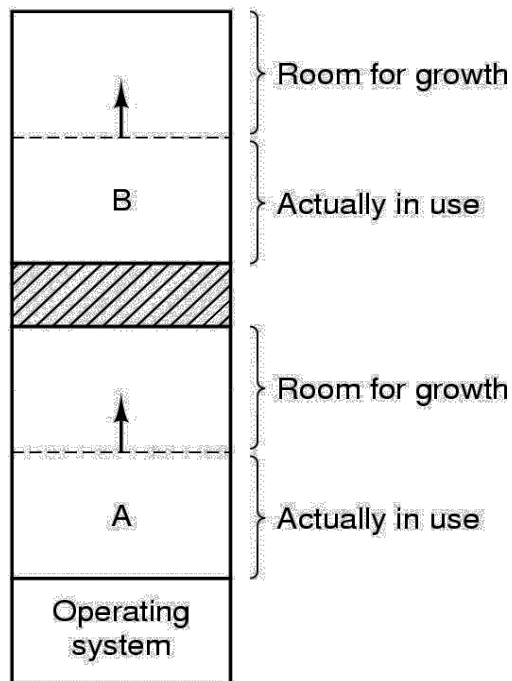
关于讨论的问题

- 运行时创建或修改的内容：栈和堆
- **交换区**：一般系统会指定一块特殊的磁盘区域作为交换空间（`swap space`），包含连续的磁道，操作系统可以使用底层的磁盘读写操作对其高效访问
- 何时需发生交换？
只要不用就换出（很少再用）；内存空间不够或有不够的危险时换出
与调度器结合使用
- 考虑进程的各种属性；不应换出处于等待I/O状态的进程

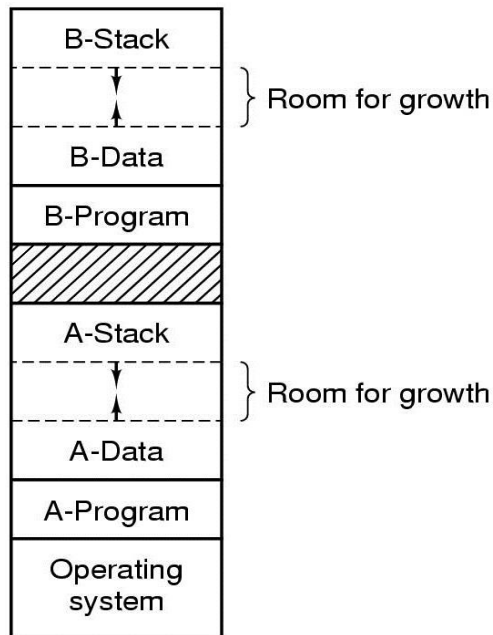
swapper

进程空间增长的困难及解决

- 数据段
- 栈段



(a)



(b)

本讲重点

- 掌握地址重定位的概念
- 掌握基本内存管理方案
- 掌握物理内存管理技术
- 了解交换技术基本思想

本周要求

重点阅读教材

第3章相关内容：3.1、3.2

重点概念

地址重定位 地址转换 地址映射 地址翻译 地址变换
逻辑地址/物理地址 虚拟地址/实地址 相对地址/绝对地址
单一连续区 固定分区 可变分区
页式 段式 段页式
物理内存管理（位图、空闲区表、空闲区链表） 伙伴系统
内存分配算法（首先适配、最佳适配、最差适配、下次适配）
内存回收算法（上相邻、下相邻、上下相邻、上下不相邻） 碎片（内碎片、外碎片） 紧缩技术 交换技术

THANKS

The End